SlotWise AI — Intelligent Warehouse Slotting & Optimization Platform

Author: Akhilesh Avinash Patil

Date: October 2026

1. Project Overview & Objectives

1.1 Background

Warehouses hold thousands of SKUs spread across many storage locations, zones and facilities. Where each SKU sits has a direct effect on how efficiently the warehouse runs. Poor placement causes:

- Longer picking distances and slower order fulfillment.
- Inefficient, frequent replenishment.
- Wasted storage capacity.
- Incompatible products (for example, flammables next to oxidizers, or food next to cleaning chemicals) stored together.
- Congested or blocked picking routes.
- Higher labor needs and avoidable safety and compliance violations.

1.2 Problem Statement

Given a warehouse network, a SKU catalogue and transaction history, decide **where each SKU should be stored**. The decision must respect every physical, thermal, hazard and policy constraint, should minimize picking effort and should explain why each recommendation was made.

1.3 Solution Summary

SlotWise AI is an end-to-end platform where every module shares one relational data model. It generates realistic, reproducible warehouse data. It then calculates demand velocity and ABC/XYZ classes, builds and validates candidate slots, and solves a constrained assignment problem. Finally, it simulates picking and replenishment on identical orders, detects exceptions, compares scenarios and presents everything through a REST API and a nine-page interactive dashboard.

1.4 Objectives

	Objective	How It Is Achieved
O1	Reproducible warehouse master and transaction data	Fixed-seed generator (seed = 20261022) with three profiles (small / medium / full) and SHA-256 manifests

	Objective	How It Is Achieved
O2	Demand intelligence	Average daily demand, FAST/MEDIUM/SLOW velocity, ABC (revenue) and XYZ (variability) classes, safety stock and reorder point
O3	Constraint-safe slot assignment	Hard-constraint filtering, then multi-factor scoring and an OR-Tools CP-SAT solver
O4	Explainability and auditability	A decision record for every selected, alternative and rejected candidate; versioned, non-destructive assignment history
O5	Measurable operational impact	Baseline vs. optimized picking on identical orders; replenishment labor simulation
O6	Exception management	Rule-based detection, severity triage, resolution workflow and automatic resolution after remediation
O7	Integration and usability	Versioned FastAPI REST API (/api/v1) and a Streamlit + Plotly dashboard
O8	Advanced intelligence	Chronological SKU demand forecasting tracked in MLflow, and a grounded LangGraph decision assistant
O9	Engineering quality	18-rule data validation, 54 automated tests, linting, Docker and CI configuration

1.5 Scope of the Datasets

Profile	Warehouses	Locations	SKUs	Order Lines	Labelled Anomalies
Small	2	500	250	5,000	21
Medium	5	2,500	1,000	25,000	51
Full	20	10,000	5,000	100,000	127

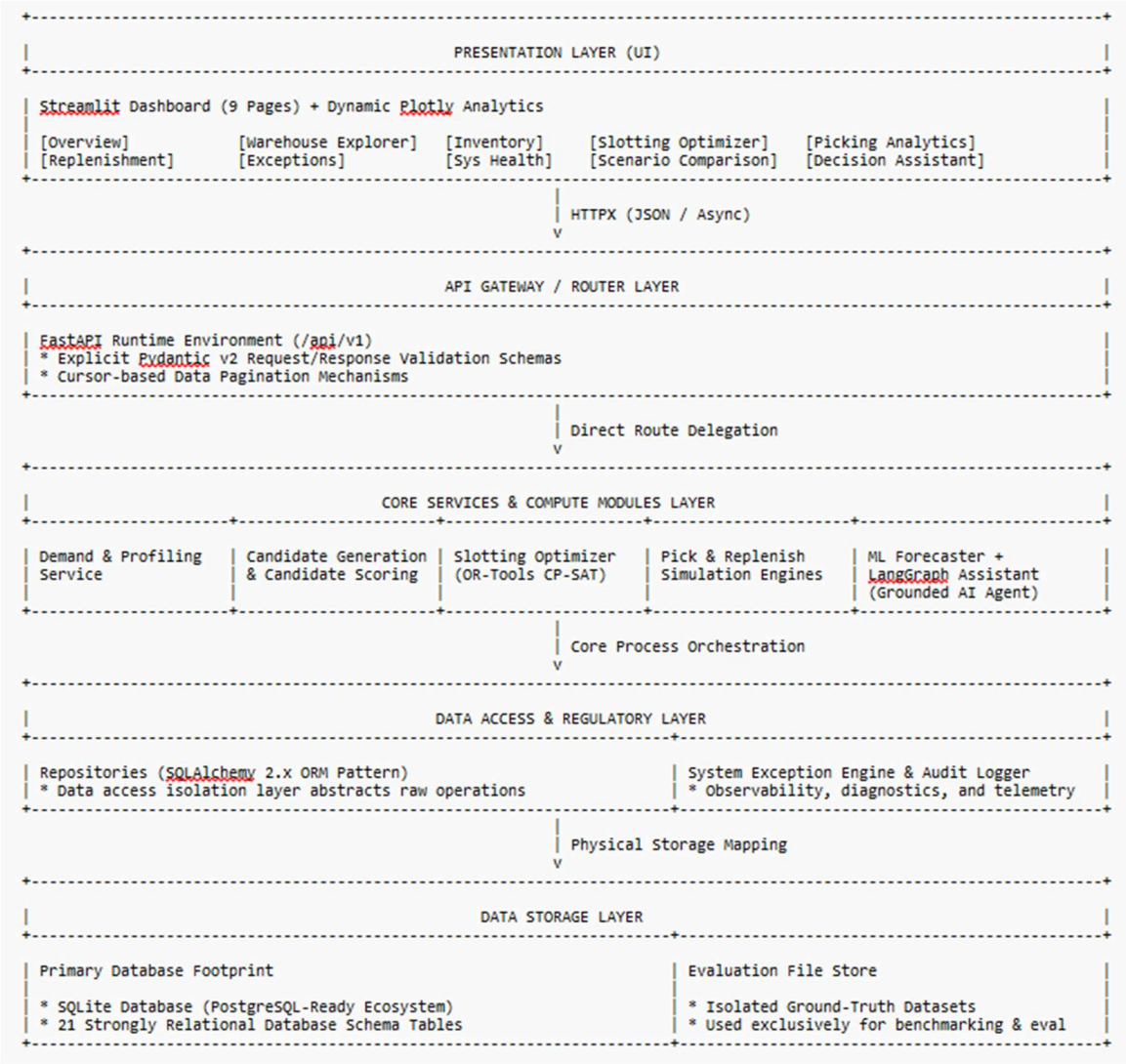
2. Architecture & System Design

2.1 Layered Architecture

SlotWise AI follows a strict layered design. API routes contain no business logic; they call the same service classes that the CLI scripts and tests use.

Layer	Responsibility	Key Components
Presentation	Interactive dashboard	Streamlit (9 pages), Plotly charts, HTTPX API client
API	Versioned REST interface, validation, error handling	FastAPI, Pydantic v2 request/response schemas, dependency injection
Service / Business	Domain logic	Demand analytics, candidate generation, scoring, exception engine, audit service, KPI service
Optimization & Simulation	Decision engines	OR-Tools CP-SAT solver, greedy + local-swap heuristic, picking route simulator, replenishment simulator
Intelligence	Forecasting and assistant	scikit-learn forecasters + MLflow; LangGraph decision assistant with a grounding firewall
Data Access	Persistence	SQLAlchemy 2.x ORM, repository layer, Alembic migrations
Storage	System of record	SQLite (default, foreign keys enforced, WAL mode); PostgreSQL-compatible schema

2.2 Architecture Diagram (description for drawing)



2.3 Data Model

The schema has **21 relational tables** with enforced foreign keys, check constraints and indexes. The core entities are:

- **Facility topology:** warehouses → zones → aisles → locations, plus hazard_classes, zone_hazard_allowances and compatibility_rules.
- **Product:** suppliers, skus, sku_profiles (time-versioned demand profiles).
- **Transactions:** orders, order_lines, pickers, pick_events, replenishments.

- **Decisions and governance:** optimization_runs, slot_assignments (versioned), assignment_reasons (one decision record per evaluated candidate), scenarios, exceptions, audit_events.

Deterministic ID conventions make records traceable, for example WH-001, SKU-000001, ORD-2026-000001, SLOT-2026-000001, OPT-2026-000001 and REP-2026-000001.

2.4 Key Design Principles

1. **Hard constraints are never overridden by scores.** Infeasible candidates are removed before scoring, and each one keeps an explicit rejection reason.

2. **History is immutable.** New assignments are PROPOSED. Approval moves the old row to SUPERSEDED (with effective_to set) and the new one to ACTIVE, with an incremented decision_version.

3. **Ground truth is isolated.** Labelled anomalies are stored in separate files that the operational exception engine never reads. They are used only for read-only precision and recall evaluation.

4. **The core is deterministic.** Optimization works without ML or any LLM; both are opt-in enhancements.

4. System Workflow

4.1 End-to-End Pipeline

Step	Stage	Description	Output
1	Data generation	Fixed-seed synthesis of facilities, SKUs, 90 days of orders, pick events and replenishments; injection of labelled anomalies	CSV/JSON/SQL exports, ground-truth files, SHA-256 manifest
2	Validation	18 rules: PK uniqueness, FK integrity, ID format, enum domains, dimensions, weights, volumes, capacity, dates, lifecycle, duplicates, profile and order arithmetic, pick and replenishment reconciliation, ground-truth separation	Validation report
3	Loading	Batched transactional load into the database	12,934 rows (small)
4	Demand profiling	ADV, standard deviation, velocity tier, ABC/XYZ class, safety stock, reorder point	sku_profiles
5	Baseline exception scan	Rule-based detection on the current layout	exceptions (21 found)
6	Candidate generation	Every SKU × location pair checked against hard constraints	29,229 feasible / 35,227 rejected
7	Scoring and optimization	Six-factor weighted score; CP-SAT assignment (greedy + local swap at large scale)	PROPOSED assignments + decision records
8	Governance approval	A manager approves or rejects the run; location loads are reconciled	ACTIVE / SUPERSEDED history
9	Post-optimization scan	Re-detection; remediated issues auto-resolved	14 auto-resolved
10	Picking simulation	Identical orders routed on the baseline and optimized layouts	Distance and time savings
11	Replenishment	Reorder-point tasks prioritized and checked	Workload and overload

Step	Stage	Description	Output
	simulation	against daily labor capacity	flags
12	Scenario and KPIs	Scenario record with deltas; live KPIs on the dashboard	scenarios, analytics endpoints

4.2 Assignment Lifecycle

PROPOSED → (approve) → ACTIVE → (superseded by a later approved run) → SUPERSEDED PROPOSED → (reject) → REJECTED

Every transition writes an audit_events record with the actor, the previous state and the new state.

4.3 Decision Assistant Workflow

1. **Intent classification:** the question is mapped to one of eight operational intents (for example picking-savings, top-exceptions or demand-surge analysis).
2. **Data retrieval:** SQL queries against live tables, scoped to the warehouse, SKU or location.
3. **Deterministic analysis:** facts, findings and recommended actions are computed in code.
4. **Synthesis:** if an LLM is configured, it writes a structured answer validated with Pydantic; otherwise a deterministic template is used.
5. **Grounding firewall:** every entity ID cited in the answer is checked against the database. If any ID is fabricated, the response falls back to the verified deterministic answer.
6. **Response:** the answer, findings, actions, source record references and an execution trace.

5. Use Case Scenarios

Use Case 1 — Re-slotting a Warehouse for Peak Demand

- **Actor:** Warehouse operations manager
- **Flow:** Open *Slotting Optimizer* → choose the warehouse scope and solver → run optimization → review proposed moves, score breakdowns and rejected alternatives → approve.
- **Outcome (small dataset):** 248 of 250 SKUs assigned and 244 SKUs relocated. The average assignment score rose from 0.6461 to 0.7451, and hard-constraint violations fell from 10 to 0.

Use Case 2 — Explaining a Placement Decision

- **Actor:** Inventory planner or auditor
- **Flow:** Search a SKU in *Inventory* → open its candidates → compare the distance, velocity, capacity, compatibility, pick-frequency and replenishment sub-scores of the chosen slot against alternatives → read the rejection reasons of infeasible slots (for example "temperature mismatch" or "exceeds max weight").
- **Outcome:** Every decision can be traced through its assignment_reasons records.

Use Case 3 — Measuring Picking Efficiency

- **Actor:** Fulfillment supervisor
- **Flow:** *Picking Analytics* → simulate the same 497 orders on both layouts.
- **Outcome:** Distance fell 23.24% (67,608.4 m → 51,893.2 m) and pick time fell 17.73%.

Use Case 4 — Handling Safety and Capacity Exceptions

- **Actor:** Safety / compliance officer
- **Flow:** *Exceptions* → filter by severity → inspect an incompatible co-location (for example flammable next to oxidizer) → apply the recommended action → mark it resolved with a note.
- **Outcome:** All 21 injected anomalies were detected (precision = recall = 1.0). Fourteen were auto-resolved after optimization.

Use Case 5 — Planning Replenishment Labor

- **Actor:** Shift planner

- **Flow:** *Replenishment* → simulate on historical demand or on ML forecasts → review tasks by priority and warehouse labor utilization → spot overloaded warehouses.
- **Outcome:** Labor demand is visible per warehouse before the shift starts.

Use Case 6 — Asking the Decision Assistant

- **Actor:** Any manager
- **Example question:** "Summarize picking distance savings across identical orders for WH-001."
- **Outcome:** A structured answer that cites only real database records. It works with no API key, using the deterministic fallback.

Negative Scenarios Verified

1. An oversized SKU that fits no location is reported as NO_COMPATIBLE_LOCATION, not forced into a slot.
2. A location loaded beyond its weight or volume limit is flagged CAPACITY_EXCEEDED.
3. Incompatible hazard or food/chemical co-location is flagged INCOMPATIBLE_COLOCATION.
4. A pick routed through a blocked aisle is flagged BLOCKED_AISLE_PICK_PATH.
5. Replenishment demand above labor capacity is flagged REPLENISHMENT_LABOR_OVERLOAD.

6. Code Structure

```
YottaFlexAI/
├── backend/
│   ├── app/
│   │   ├── main.py            # FastAPI app factory, /health, CORS, error handlers
│   │   ├── config.py          # YAML profile + environment settings
│   │   ├── database.py        # Engine, session factory, SQLite FK/WAL pragmas
│   │   ├── dependencies.py    # DI, pagination, domain errors
│   │   ├── models/            # 21-table SQLAlchemy ORM + enums
│   │   ├── schemas/           # Pydantic v2 request/response models
│   │   ├── repositories/      # Data-access layer
│   │   ├── services/          # Data generator, loader, demand analytics, exception engine, audit
│   │   ├── validators/        # 18-rule dataset validator
│   │   ├── optimization/      # Candidate generation, scoring, CP-SAT / greedy solver
│   │   ├── simulation/        # Picking route and replenishment simulators
│   │   ├── analytics/         # KPI service, ground-truth evaluation, scenarios
│   │   ├── ml/                # Demand forecaster + MLflow tracking
│   │   ├── agents/            # LangGraph assistant, LLM adapter, grounding firewall
│   │   ├── api/v1/            # 14 routers (warehouses, skus, optimization, ...)
│   │   └── utils/             # Deterministic IDs, checksums, time helpers
│   ├── alembic/               # Database migrations
│   └── tests/                 # Phase 1–7 pytest suites
├── frontend/
│   ├── app.py                 # Streamlit launchpad
│   ├── components/            # API client, theme, sidebar, charts, assistant panel
│   └── pages/                 # 01_Overview … 09_System_Health
```

```
├── scripts/                 # generate, validate, load, reproduce, simulate, train, audit, benchmark
├── configs/                 # default / small / medium / full YAML profiles
├── data/                    # generated, ground_truth, exports, manifests
├── artifacts/               # models, mlruns, reports, benchmarks
├── docs/                    # architecture, data dictionary, business rules, API, testing ...
├── Dockerfile · docker-compose.yml · .github/workflows/ci.yml
├── Makefile · requirements.txt · pyproject.toml · alembic.ini
└── README.md
```

Module Responsibilities

Module	**Main Classes**	**Purpose**
services/data_generator.py	SyntheticDataGenerator	Fixed-seed data and labelled anomalies
validators/dataset_validator.py	DatasetValidator	18 integrity and business rules
services/demand_analytics.py	DemandAnalyticsService	ADV, velocity, ABC/XYZ, safety stock, ROP
optimization/solver.py	SlottingOptimizerService	Constrained assignment, approval and rejection
simulation/picking_simulator.py	PickingSimulationService	Route distance and time comparison
simulation/replenishment_simulator.py	ReplenishmentSimulationService	Labor workload simulation
services/exception_engine.py	ExceptionDetectionService	Detection and auto-resolution
analytics/kpi_service.py	AnalyticsKPIService	Live KPIs and ground-truth

Module	Main Classes	Purpose
		F1
ml/forecaster.py	DemandForecastingService	Chronological forecasting + MLflow
agents/assistant_service.py	DecisionAssistantService	Grounded LangGraph assistant

7. API Endpoints

GThe base path is /api/v1, except for /health. There are 38 versioned endpoints plus the health check, all documented in Swagger at /docs. List endpoints support filtering and pagination (page, page_size).

Method	Endpoint	Description
GET	/health	Service and database health
System		
GET	/api/v1/system/status	System, dataset and configuration status
GET	/api/v1/system/audit-events	Immutable audit trail (paginated)
Data Pipeline		
POST	/api/v1/data/generate	Generate a dataset profile
POST	/api/v1/data/validate	Run the 18-rule validator
GET	/api/v1/data/manifest	Dataset manifest and checksums
Warehouses & Locations		
GET	/api/v1/warehouses	List warehouses
GET	/api/v1/warehouses/{warehouse_id}	Warehouse details
GET	/api/v1/locations	List / filter locations
GET	/api/v1/locations/{location_id}	Location details and current load

Method	Endpoint	Description
SKUs		
GET	/api/v1/skus	Search / filter SKUs
GET	/api/v1/skus/{sku_id}	SKU details
GET	/api/v1/skus/{sku_id}/profile	Demand profile (ADV, velocity, ABC/XYZ, ROP)
GET	/api/v1/skus/{sku_id}/candidates	Ranked feasible and rejected candidate slots
Slotting Optimization		
POST	/api/v1/optimization/run	Run optimization (scope, strategy, ML option)
GET	/api/v1/optimization/runs	List runs
GET	/api/v1/optimization/runs/{run_id}	Run details and KPIs
GET	/api/v1/optimization/runs/{run_id}/assignments	Proposed / active assignments
POST	/api/v1/optimization/runs/{run_id}/approve	Approve → ACTIVE
POST	/api/v1/optimization/runs/{run_id}/reject	Reject proposals
GET	/api/v1/optimization/assignments/history	Versioned assignment history
Picking & Replenishment		
POST	/api/v1/picking/simulate	Baseline vs. optimized routes on identical orders
GET	/api/v1/picking/analytics	Picking KPIs

Method	Endpoint	Description
POST	/api/v1/replenishment/simulate	Labor workload simulation
GET	/api/v1/replenishment/analytics	Replenishment KPIs
Exceptions		
GET	/api/v1/exceptions	List / filter exceptions
POST	/api/v1/exceptions/scan	Run detection
GET	/api/v1/exceptions/{exception_id}	Exception details
PATCH	/api/v1/exceptions/{exception_id}/resolve	Resolve or dismiss with a note
Scenarios & Analytics		
POST	/api/v1/scenarios	Create and evaluate a scenario
GET	/api/v1/scenarios	List scenarios
GET	/api/v1/scenarios/{scenario_id}/comparison	Baseline vs. optimized deltas
GET	/api/v1/analytics/overview	Executive KPIs + ground-truth evaluation
GET	/api/v1/analytics/warehouse/{warehouse_id}	Facility analytics
GET	/api/v1/analytics/optimization	Optimization history and score components
ML & Assistant		
GET	/api/v1/ml/status	Model status and evaluation summary
POST	/api/v1/ml/train	Train and evaluate forecasters

Method	Endpoint	Description
GET	/api/v1/ml/forecasts	SKU forecasts (filter, sort, paginate)
POST	/api/v1/assistant/query	Grounded decision assistant

Error handling: 404 for unknown entities, 422 for validation errors and 400 for invalid state transitions (for example approving an already-approved run). All errors use a consistent JSON body.

8. Conclusion

The SlotWise AI delivers a complete, working and reproducible warehouse slotting platform. Data generation, validation, optimization, simulation, exception management, analytics, forecasting and an assistant all run on one shared data model, and the same services back the CLI, the API and the dashboard.

Key Results (small dataset, verified runs)

- **−23.24% picking distance** and **−17.73% picking time** on the same 497 orders.
- **Hard-constraint violations reduced from 10 to 0**, and average assignment score up **15.3%**.
- **100% exception detection** against isolated ground truth (21 / 21, 0 false positives).
- **Full explainability:** 2,098 decision records, including 750 rejected candidates with reasons.
- **Reproducibility:** byte-identical outputs across runs (20 files, SHA-256 verified).
- **Quality:** 54 automated tests passing, 39 / 40 independent raw-SQL integrity checks passing (one expected warning for the 2 oversized SKUs), and a clean lint report.